PROYECTO 2 · CC3084 DATA SCIENCE

Reconocimiento de deletreo manual en ASL con aprendizaje profundo

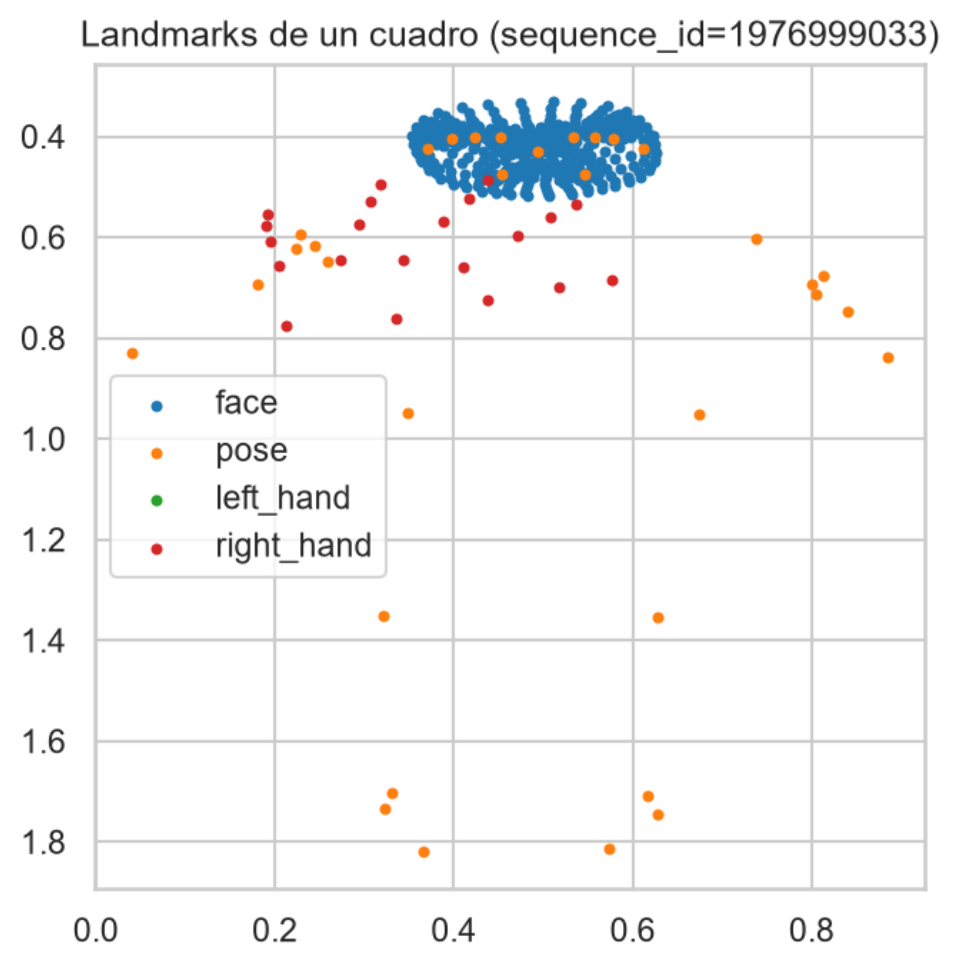
Objetivo: transcribir a texto secuencias de landmarks de las manos (deletreo manual en ASL), incluso de personas que el modelo nunca vio al entrenar.

María José Girón · Daniela Ramírez · Leonardo Mejía · Cindy Gualim · José Donado

Universidad del Valle de Guatemala · 19 de octubre de 2026

EL RETO Y LOS DATOS

No vemos video: vemos un esqueleto que se mueve



Landmarks: puntos (x, y, z) que MediaPipe ubica en cada cuadro. 21 por mano, más pose y rostro.

Se deletrean 5-6 letras por segundo: una letra no es una foto, es un movimiento en el tiempo.

Los dedos ya se acomodan a la letra siguiente (coarticulación): hay que mirar la secuencia completa.

Datos esenciales: 67,208 secuencias, 94 participantes, frases de 1 a 31 caracteres.

CONTEXTO · ENTREGA ANTERIOR

Recordatorio: lo que descubrimos en el EDA

El problema

- 12-35 % del discurso en ASL es deletreo: nombres, siglas y términos técnicos.
- 5-6 letras por segundo: muy rápido y con coarticulación.
- Letras parecidas (M, N, T) y mucha variación entre personas, encuadre y oclusión.

Hallazgo del EDA → decisión de modelado

La mano derecha falta en 54.5 % de los cuadros	**Forward-fill + máscara de detección**
94 participantes, de 1 a 1,535 secuencias cada uno	**División por participante**
Secuencias de 3 a 751 cuadros (mediana 147)	**Recorte a 384 cuadros + CTC**
59 caracteres muy desbalanceados (e, a, o vs. j, q, z)	**Reportar el error por carácter**

CÓMO FUNCIONA

De movimiento a texto: el sistema paso a paso

Video

Grabación con la cámara de un teléfono

Landmarks

MediaPipe: coordenadas por cuadro

Preprocesamiento

Limpieza, normalización, máscara

Modelo

CNN-BiGRU o Transformer: probabilidades por paso

Texto

Decodificación CTC greedy

¿Cómo se entrena sin saber en qué cuadro empieza cada letra? CTC

La etiqueta es la frase completa, no una letra por cuadro. CTC suma todos los alineamientos posibles que producen esa frase.

Al predecir: se elige el símbolo más probable por paso, se fusionan repeticiones y se borran los “blank”.

Ejemplo ilustrativo: _ b b _ u u _ g g s _ → bugs

PREPARACIÓN DE DATOS

De 1,630 columnas por cuadro a 227 entradas útiles

1. Manos faltantes

La mano derecha falta en 54 % de los cuadros. Se rellena con el último valor válido (forward-fill) y se añade una máscara de detección por mano.

2. Normalización

Manos relativas a la muñeca; pose relativa a los hombros. El modelo ve la forma, no dónde está sentada la persona.

3. Descarte

Se quitó el rostro (86 % de columnas) y 119 secuencias con una mano detectada en <10 % de cuadros: 1,997 → 1,878.

4. Entrada final

227 valores por cuadro (225 coordenadas + 2 máscaras). Máximo 384 cuadros (38 secuencias recortadas); etiquetas de 59 caracteres + blank.

El forward-fill no recupera el movimiento perdido; por eso la máscara le avisa al modelo qué es observado y qué es imputado.

MÉTODO DE EVALUACIÓN

División por participante: nadie se repite entre particiones

1,315

secuencias de entrenamiento
64 participantes (70 %)

295

secuencias de validación
14 participantes (15 %) · elegir configuración

268

secuencias de prueba
14 participantes (15 %) · reservado al final

¿Por qué separar por participante?

Cada persona deletrea con su propio estilo y velocidad. Si una misma persona estuviera en entrenamiento y prueba, el modelo podría memorizarla.

Así se evita la fuga de información y se mide lo que importa: generalizar a personas nuevas (semilla 42).

Métrica principal: CER = (sustituciones + omisiones + inserciones) / caracteres de referencia. Menor es mejor.

MODELOS Y AJUSTES

Dos arquitecturas, mismo protocolo (CTC, AdamW, 60 épocas)

CNN-BiGRU

Convoluciones 1D (patrones locales + reducción temporal) y GRU bidireccional (contexto antes y después).

Cambios probados →	CER de validación
Base (192 can., 128 GRU, dropout 0.3)	0.596
Mayor capacidad (256 / 192) ✓	0.552
Stride 4	0.554
Dropout 0.5	0.797

Transformer

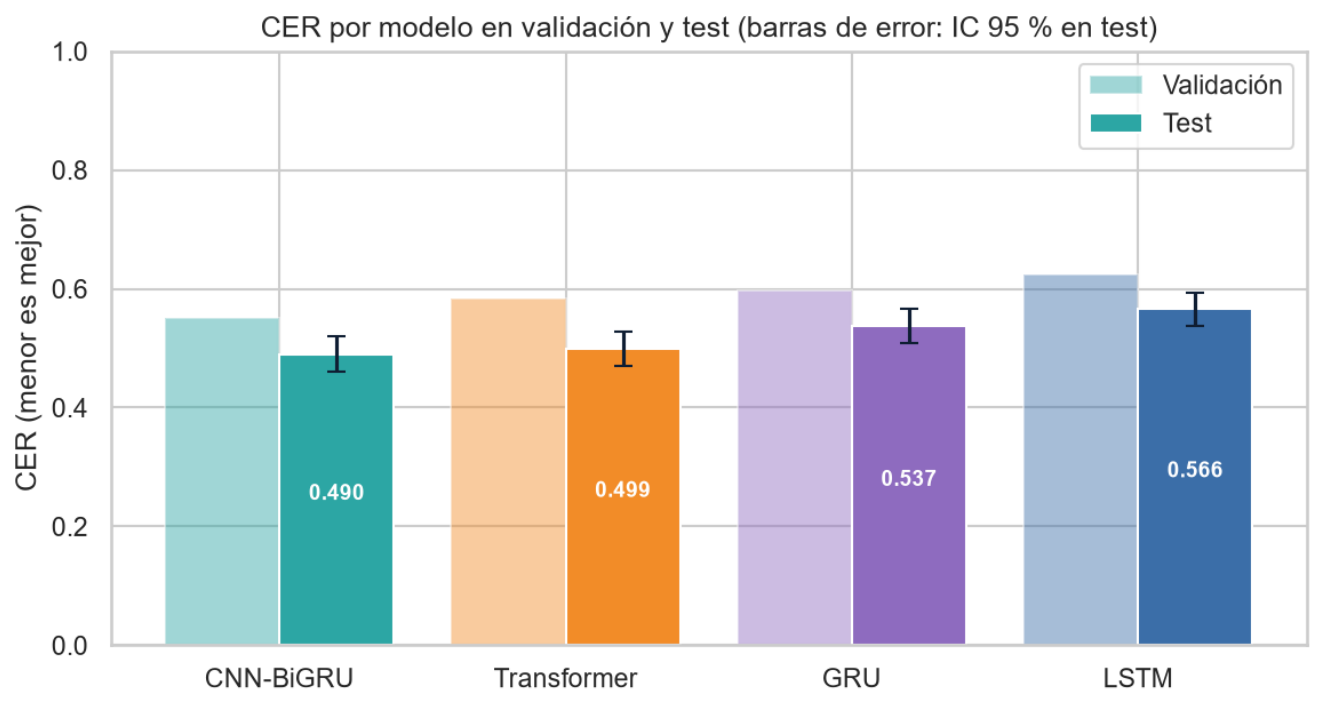
Convoluciones iniciales + encoder con atención: cada cuadro se relaciona con todos los demás.

Cambios probados →	CER de validación
Base (dim. 128, 3 capas, dropout 0.3)	0.680
Dropout 0.1 ✓	0.584
Stride 4	0.588
Mayor ancho 192	0.626
Learning rate 0.003	0.640
6 capas	0.654

✓ = configuración elegida. Más capacidad ayudó al CNN-BiGRU; en el Transformer funcionó mejor regularizar menos que agrandar.

RESULTADOS PRINCIPALES

Error casi empatado; el costo no



CNN-BiGRU

0.4898
CER prueba

1.70 M
parámetros

47.66 min
entrenamiento

Transformer

0.4991
CER prueba

0.63 M (≈37 %)
parámetros

17.81 min
entrenamiento

CNN-BiGRU tiene el menor CER de prueba (diferencia de 0.0094), pero Wilcoxon da $p \approx 0.21$: no hay diferencia significativa.

El Transformer usa 37 % de los parámetros, entrena en 17.81 min vs. 47.66 min y evalúa más rápido (1.77 vs. 5.34 ms por secuencia).

OTRAS PRUEBAS

Líneas base recurrentes y aumentación de datos

Modelos de referencia (CER de prueba)

Modelo	CER	Parámetros
CNN-BiGRU	0.4898	1.70 M
Transformer	0.4991	0.63 M
GRU	0.5366	0.50 M
LSTM	0.5658	0.65 M

Las recurrentes unidireccionales (solo ven el pasado) quedan por detrás de los dos modelos principales; no se puede atribuir la mejora solo al contexto bidireccional, porque también cambian arquitectura e hiperparámetros.

¿Ayuda la aumentación de datos?

0.493

sin aumentación
(CER de prueba)

0.506

con aumentación
(CER de prueba)

Misma CNN-BiGRU (stride 4), 60 épocas, una semilla. Redujo el sobreajuste (brecha de pérdida 0.98 → 0.72), pero no el CER: no hay evidencia de que ayude con este presupuesto de entrenamiento.

Los modelos finales se entrenaron sin aumentación.

QUÉ REVELAN LOS ERRORES

Los errores: letras omitidas y manos sin detectar

76 % / 68 %

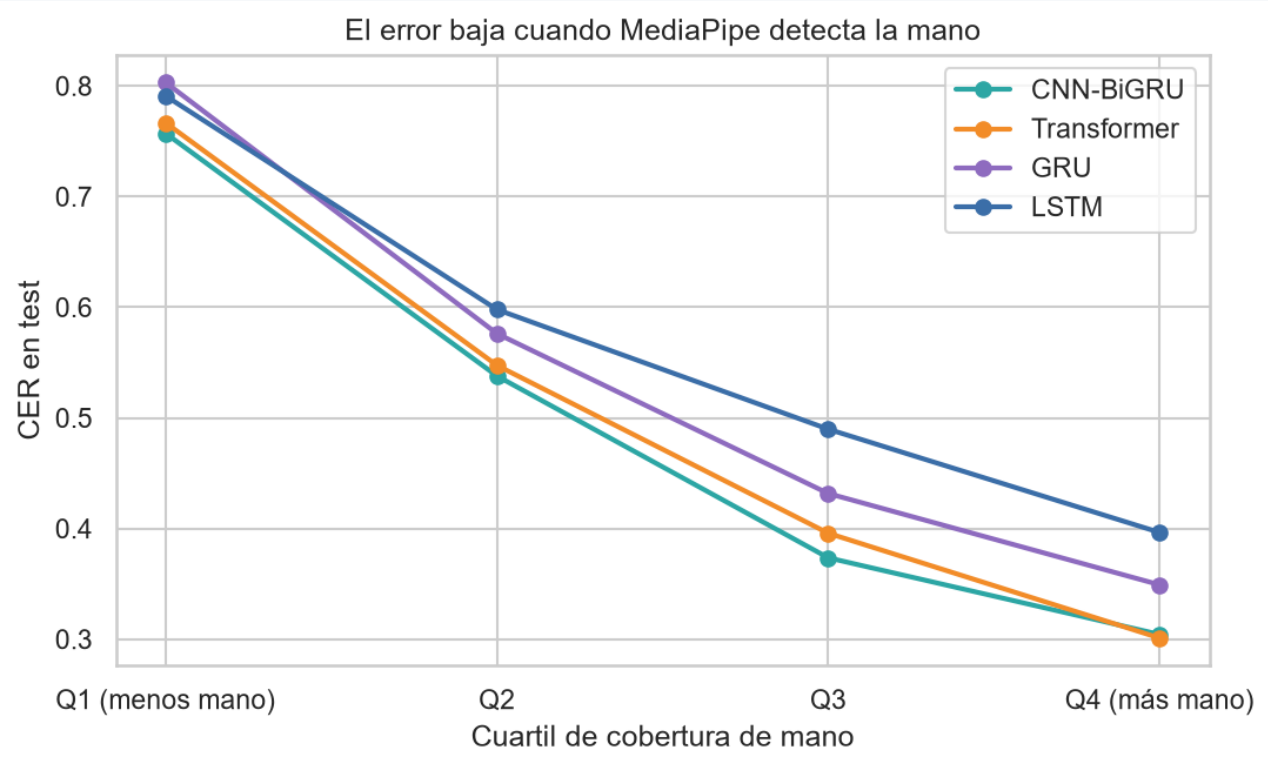
de los errores son caracteres omitidos (CNN-BiGRU / Transformer)

ρ ≈ –0.72 / –0.74

más manos detectadas → menor CER (Spearman; asociación, no causalidad)

q · x · z · j

caracteres raros en entrenamiento: se fallan casi siempre



Lo que limita las predicciones es la información de entrada (manos no detectadas) y los caracteres poco frecuentes, más que la arquitectura.

LA APLICACIÓN

Ingresar una secuencia, ver la predicción y explorar el rendimiento

① Predecir una frase

Todos Uno

3. Resultado

Frase real: `bugs.launchpad.net`

CNN-BiGRU — CER **0.278** · 34 ms

`bu-s-launchpad-nes`

Transformer — CER **0.278** · 7 ms

`bugs-launchpad-net`

correcto sustitución inserción letra omitida

② Rendimiento de los modelos

Rendimiento de los modelos

Mejor modelo en test	CER test	Caracteres correctos (≈ 1 − CER)
CNN-BiGRU	0.490	51 %

Modelo	CER test	CER validación	Parámetros (M)	Entrenamiento (min)	ms por secuencia	Épocas
CNN-BiGRU	0.490	0.552	1.70	47.66	5.34	60
Transformer	0.499	0.584	0.63	17.81	1.77	60

CER por modelo

Mostrar

Character error rate en validación y en test (menor es mejor). Validación se usó para elegir hiperparámetros; test solo se usó al final.

1

Se elige una secuencia de prueba o se sube un parquet nuevo, que se preprocesa igual que al entrenar.

Cada carácter se colorea según la alineación: verde correcto · rojo sustituido · subrayado insertado · tachado omitido.

El CER solo se puede calcular si existe una transcripción de referencia; con un archivo nuevo sin ella solo se muestra la predicción.

CONCLUSIONES Y LÍMITES

¿Qué modelo elegimos y qué tan lejos estamos?

Conclusiones

Elegimos el Transformer para el prototipo: error casi igual, 37 % de parámetros y 2.7× menos tiempo de entrenamiento.

Si lo único que importa es el menor error: CNN-BiGRU (0.4898 vs. 0.4991).

Agrandar la arquitectura no garantizó mejorar; ajustar regularización y stride sí.

Más manos detectadas, menos error: la calidad de la entrada pesa más que la arquitectura.

Límites

Con CER ≈ 0.49 aún no es un transcriptor confiable: un error puede cambiar el significado.

La prueba cubre solo 14 participantes: hay que ser cautos al generalizar.

Se usaron 2 archivos (1,997 de 67,208 secuencias); tiempos medidos en CPU; sin modelo de lenguaje.

Próximos pasos: más datos, más épocas con aumentación y mejor decodificación.



Gracias

Preguntas y comentarios

María José Girón · Daniela Ramírez · Leonardo Mejía · Cindy Gualim · José Donado

CC3084 Data Science · Universidad del Valle de Guatemala

github.com/DufreyM/CC3084-Proyecto2